

Reporte — Proyecto 1: Uso de un protocolo existente (MCP)

Universidad del Valle de Guatemala · Facultad de Ingeniería Departamento de Ciencias de la Computación · CC3067 Redes

Estudiante	Pablo Cabrera
Carné	231156
Proyecto	Chatbot con Model Context Protocol (MCP) — trabajo individual
Caso de uso del servidor propio	Cadena de farmacias “FarmaValle”
Lenguaje	Python 3
LLM	Google Gemini (vía API de Gemini)
Repositorio	https://github.com/pablouwunya2021/chatbot_redes-
Servidor remoto (Cloud Run)	https://pharmacy-mcp-740391845268.us-central1.run.app/mcp

El protocolo MCP se implementó **manualmente sobre JSON-RPC 2.0**, sin usar SDKs de MCP ni FastMCP, tal como exige el enunciado. El SDK de Google Gemini se utiliza únicamente para hablar con el LLM a nivel de su API.

1. Introducción

Los *Large Language Models* (LLM) no pueden, por sí solos, interactuar con el mundo real: no leen archivos, no consultan inventarios ni ejecutan acciones. El **Model Context Protocol (MCP)**, propuesto por Anthropic en noviembre de 2024, es un estándar abierto que resuelve esto proveyendo **herramientas** a los LLM de forma **interoperable**: un desarrollador define una herramienta una sola vez y cualquier LLM compatible puede usarla. MCP usa **JSON-RPC 2.0**, un protocolo de la **capa de aplicación** de los modelos OSI y TCP/IP.

Este proyecto implementa un **chatbot (anfitrión / host)** que conecta un LLM con varios **servidores MCP**, tanto oficiales como uno propio, y que funciona igual con servidores **locales** (stdio) y **remotos** (HTTP en la nube).

2. Objetivos cumplidos

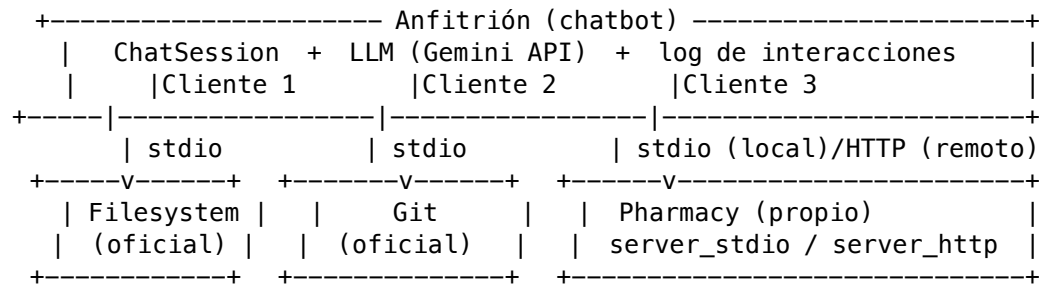
- Implementar el protocolo MCP con base en los estándares (JSON-RPC 2.0).
- Comprender el propósito y los servicios de MCP (lifecycle, discovery, calls).
- Implementar servidores MCP **locales** y **remotos**.
- Interactuar con un LLM a nivel de la **API**.

3. Arquitectura MCP y su relación con este proyecto

MCP define tres actores:

Actor	Definición	En este proyecto
Servidor	expone y ejecuta las herramientas	Filesystem y Git (oficiales) + Pharmacy (propio)
Cliente	mantiene la conexión con un servidor y sabe cómo usarlo	MCPServerConnection (uno por servidor) en <code>mcp_client.py</code>

Actor	Definición	En este proyecto
Anfitrión	app de IA que coordina múltiples clientes	el chatbot (ChatSession en chatbot.py)



4. Implementación

4.1 Cliente MCP manual (JSON-RPC 2.0)

src/host/mcp_client.py implementa a mano:

- **Enmarcado JSON-RPC 2.0:** contador de id, y las tres formas de mensaje (petición con id+method, notificación con method sin id, respuesta con id+result/error).
- **Ciclo de vida MCP:** initialize -> el servidor responde versión y capacidades -> el cliente envía la notificación notifications/initialized.
- **Descubrimiento:** tools/list.
- **Invocación:** tools/call.
- **Dos transportes** que hablan los mismos mensajes:
 - StdioTransport: lanza el servidor como proceso hijo e intercambia JSON delimitado por saltos de línea por stdin/stdout.
 - HttpTransport: hace POST de cada mensaje a un endpoint HTTP; maneja respuestas application/json o SSE, y la cabecera Mcp-Session-Id.

4.2 Anfitrión / chatbot

src/host/chatbot.py conecta todos los servidores, agrega sus herramientas (nombradas servidor__herramienta para evitar colisiones), mantiene el **contexto** de la conversación (lista messages) y **enruta** cada llamada de herramienta que pide el LLM al cliente correcto. src/host/llm.py traduce las herramientas MCP al formato de *function declarations* de Gemini y ejecuta el bucle *tool-use*.

4.3 Servidores MCP oficiales (Requisito #4)

- **Filesystem** (@modelcontextprotocol/server-filesystem, vía npx), 14 herramientas, en una carpeta *sandbox*.
- **Git** (mcp-server-git, vía uvx), 12 herramientas.

El script src/demo_git_filesystem.py demuestra el escenario del enunciado: **crear un repositorio, crear un README, agregarlo y hacer commit**, mostrando después el git_log. *Hallazgo real:* la versión actual de mcp-server-git ya **no incluye** la herramienta git_init, por lo que la **creación** del repo se hace con git init y el resto (git_add, git_commit, git_log, git_status) a través del servidor MCP oficial.

4.4 Servidor MCP propio — Pharmacy (Requisito #5)

Caso de uso a nivel de industria: una **cadena de farmacias**. El cliente describe síntomas y el bot recomienda productos de venta libre (OTC), consulta inventario y realiza pedidos. La lógica está en core.py

(compartida por ambos transportes) y el catálogo/inventario en `data.py`.

Especificación resumida (detalle completo en [SERVER_SPEC.md](#)):

- Identidad: `pharmacy-mcp v1.0.0`, protocolo MCP 2025-06-18.
- Métodos JSON-RPC: `initialize`, `notifications/initialized`, `ping`, `tools/list`, `tools/call`.
- **Herramientas (7)**: `list_stores`, `search_medications`, `get_medication_info`, `recommend_for_symptoms`, `check_inventory`, `place_order`, `get_order_status`.
- **Endpoints (transporte HTTP)**: `POST /mcp` (JSON-RPC), `GET /healthz`, `GET /`. Cabecera opcional `X-API-Key` para autenticación.
- **Seguridad clínica**: cada respuesta clínica incluye un *disclaimer* y los síntomas de emergencia (“red-flag”) se derivan a atención profesional en vez de recomendar un producto.

4.5 Servidor MCP remoto (Requisito #6)

El **mismo** `core.py` se expone por HTTP en `server_http.py` (FastAPI/uvicorn como simple servidor HTTP; el protocolo sigue siendo manual). Se empaqueta con `deploy/Dockerfile` y se despliega en **Google Cloud Run** (o Cloudflare); ver [deploy/DEPLOY.md](#). El chatbot lo usa **igual** que al local: `python src/host/cli.py --remote`. Lo único que cambia es el transporte (stdio -> HTTP).

El servidor se desplegó efectivamente en Google Cloud Run y quedó operativo en `https://pharmacy-mcp-740391845268.us-central1.run.app/mcp`. Se verificó de extremo a extremo: el cliente hecho a mano completó el *handshake*, listó las 7 herramientas y ejecutó varias `tools/call` (incluida la creación de un pedido) contra la instancia en la nube, demostrando que el chatbot consume el servidor remoto exactamente igual que el local.

5. Funcionalidades (mapa con la rúbrica)

Requisito	Estado	Evidencia
1. Conexión con LLM por API	Sí	<code>llm.py</code> , chat general
2. Contexto en la sesión	Sí	lista <code>messages</code> preservada entre turnos
3. Log de interacciones MCP	Sí	<code>logs/mcp_interactions.jsonl</code> , /log, panel web
4. Filesystem + Git oficiales	Sí	<code>demo_git_filesystem.py</code>
5. Servidor MCP local propio	Sí	<code>server_stdio.py</code> (pharmacy)
6. Servidor MCP remoto	Sí	<code>server_http.py</code> + <code>deploy/</code>
7. Análisis con Wireshark	Sí	§6 y <code>docs/wireshark/</code>
8. Especificación del servidor	Sí	<code>SERVER_SPEC.md</code> y §4.4
9. Análisis por capas	Sí	§6.2
10. Conclusiones	Sí	§8
Extra: UI (15%)	Sí	CLI Rich + web FastAPI

6. Análisis de la comunicación con Wireshark (Requisitos #7 y #9)

Procedimiento completo y reproducible en [docs/wireshark/WIRESHARK_ANALYSIS.md](#). Se captura contra el servidor remoto ejecutándose en **HTTP plano local** (Cloud Run cifra con TLS; para leer el contenido se usa HTTP plano o `SSLKEYLOGFILE`). La captura se realizó sobre la interfaz de *loopback* filtrando `tcp.port == 8000` (archivo `docs/wireshark/capture.pcapng`, 100 paquetes capturados).

Figura 1. Paquete del mensaje `initialize` capturado en *loopback*. En el panel superior se ve el **payload de aplicación**: el mensaje JSON-RPC 2.0 `{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocol":"06-18",...,"clientInfo":{"name":"uvg-redes-chatbot",...}}}`, es decir la **petición de sincronización inicial** del *handshake* MCP. En el panel inferior se observa el anidamiento por capas de este

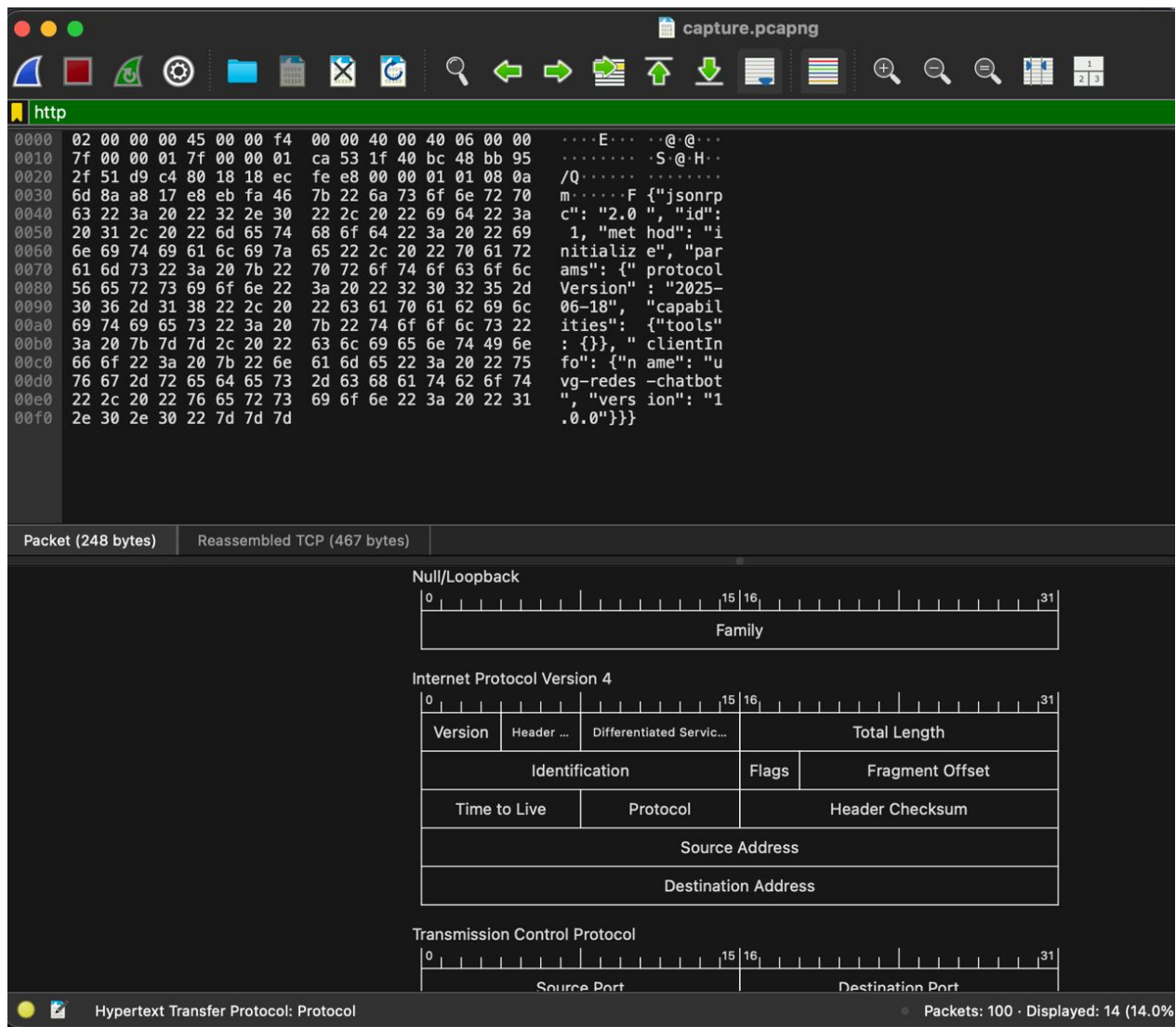


Figure 1: Figura 1

mismo paquete: **Null/Loopback** (enlace) -> **IPv4** con origen y destino 127.0.0.1 (red) -> **TCP** puerto 8000 (transporte) -> **HTTP** que transporta el JSON (aplicación). El paquete mide 248 bytes y el mensaje completo se reensambla en 467 bytes de TCP, confirmando la segmentación descrita en §6.2.

6.1 Clasificación de mensajes JSON-RPC

Una sesión típica son **13 mensajes** = 6 pares petición/respuesta + 1 notificación:

Mensajes	Clasificación	Cómo se identifican en Wireshark
initialize (req), su result, y notifica- tions/initialized	Sincronización (handshake)	negocian versión y capacidades antes de trabajar
initialize, tools/list, tools/call	Petición / request	el JSON tiene method y id ; en HTTP son POST /mcp
notifications/in los result (id 1..6)	Notificación Respuesta	tiene method sin id ; el servidor responde 202 sin cuerpo tienen id y result sin method; en HTTP son 200 OK; el id empareja con su petición

Filtros útiles: `http.request.method == "POST"` (peticiones/notificaciones), `http.response.code == 200` (respuestas), `http.response.code == 202` (ack de la notificación), `json.key == "method"` vs `json.key == "result"`.

6.2 Qué sucede en cada capa

- **Aplicación (HTTP + JSON-RPC/MCP):** el cuerpo es un mensaje JSON-RPC 2.0 (MCP), transportado por HTTP/1.1 (POST /mcp, Content-Type: application/json, cabeceras MCP-Protocol-Version, Mcp-Session-Id, X-API-Key; respuestas 200/202). Aquí vive la distinción sincronización/petición/ respuesta.
- **Transporte (TCP, puerto 8000):** *handshake* de 3 vías SYN/SYN, ACK/ ACK; entrega **fiable y ordenada**; las respuestas grandes (p. ej. tools/list) se segmentan y Wireshark las reensambla; el puerto destino identifica el proceso servidor.
- **Red (IP):** enrutamiento best-effort; en loopback 127.0.0.1 <-> 127.0.0.1; contra Cloud Run, IP pública del servicio. TTL e identificación IP.
- **Enlace:** en loopback, encapsulado Null/Loopback (sin MAC real); en una LAN sería Ethernet/Wi-Fi con MAC origen/destino (ARP) y un MTU (~1500 B) que obliga a segmentar en TCP.

7. Dificultades y lecciones aprendidas

- **Implementar MCP a mano** obliga a entender el ciclo de vida real: sin la notificación notifications/initialized varios servidores no responden a tools/list. Lección: MCP es *stateful*; el handshake importa.
- **stdio es frágil con la salida estándar:** cualquier print del servidor a *stdout* corrompe el flujo JSON-RPC. Solución: todo log del servidor va a *stderr* (y se drena en un hilo aparte para no bloquear el *pipe*).
- **Emparejar respuestas por id:** los servidores pueden intercalar notificaciones; el cliente lee hasta encontrar el id esperado y registra el resto en el log.
- **git_init ya no existe** en el servidor Git oficial: hubo que ajustar el escenario (crear el repo con git init y usar MCP para el resto).
- **TLS oculta el payload:** capturar Cloud Run directamente muestra solo *Application Data* cifrada; por eso se captura en HTTP plano local o se usa SSLKEYLOGFILE. Lección práctica sobre seguridad en la capa de transporte.

- **Namespacing de herramientas:** con 33 herramientas de 3 servidores, nombrar `servidor__herramienta` evita colisiones y permite enrutar la llamada.

8. Conclusiones y comentarios (Requisito #10)

- MCP resuelve un problema real de **interoperabilidad**: definir una herramienta una vez y usarla desde cualquier LLM. Separar *qué* hace la herramienta de *cómo* la invoca el modelo es una abstracción muy poderosa.
- **JSON-RPC 2.0 sobre distintos transportes** (stdio y HTTP) demuestra que el protocolo de aplicación es independiente del transporte: el mismo `core.py` sirvió local y en la nube sin cambios.
- El análisis con Wireshark hizo tangible la **pila de protocolos**: un simple “recomiéndame algo para la fiebre” viaja como JSON-RPC -> HTTP -> TCP -> IP -> enlace, con su *handshake* y reensamblado.
- Construir el protocolo a mano, en lugar de usar un SDK, dejó claro por qué el estándar toma cada decisión (*handshake*, *id*, notificaciones, capacidades).
- Como mejora futura: persistir pedidos en una base de datos, soportar el transporte *Streamable HTTP* con streaming SSE completo, y añadir autenticación OAuth para el servidor remoto.

9. Referencias

- JSON-RPC 2.0 — <https://www.jsonrpc.org/specification>
- MCP Architecture — <https://modelcontextprotocol.io/docs/learn/architecture>
- MCP Specification (2025-11-25) — <https://modelcontextprotocol.io/specification/2025-11-25>
- MCP servers — <https://github.com/modelcontextprotocol/servers>
- Tutorial MCP remoto en Google Cloud Run — <https://cloud.google.com/blog/topics/developers-practitioners/build-and-deploy-a-remote-mcp-server-to-google-cloud-run-in-under-10-minutes>